Y会場（AB-1）

大規模言語モデルを用いた行間を埋める自動証明

潘 俊洋 & 松崎 拓也（東京理科大）

概要：既存の LLM 証明器を形式証明スケッチの補完に適応させ，行間補完モデルを構築した．結果として $\boldsymbol{pass@32 = 95.08\%}$ を達成した．

1. Lean 4

数学の証明をプログラム言語で記述し，その正しさを計算機で厳密にチェックするシステム

命題：全ての自然数 n, m に対して，n が偶数ならば $m * n$ も偶数である．

```
theorem my_example : ∀ m n : Nat, Even n → Even (m * n) := by
  rintro m n ⟨ k, hk ⟩ -- m, n を自然数とし，n は自然数 k で n = k + k と表せるという仮定 hk を導入する．
  use m * k            -- m * n = (m * k) + (m * k) を示すことで，m * n が偶数であることを示す．
  rw [ hk ]            -- n に仮定 hk : n = k + k を代入
  ring                 -- m * (k+k) = (m * k) + (m * k) より明らか
```

→ Lean 4 コンパイルで検証 → 証明完了

2. 背景 と 目的

- Draft, Sketch, and Prove（DSP ; Jiang +, ICLR'23）は，自然言語証明から部分的な形式証明（形式スケッチ）を生成し，その未証明の部分（行間）を補完することで，完全な形式証明を得る枠組みである．
- しかし，形式スケッチ中の行間を補完するタスクに特化した学習済み証明モデルは十分に整備されていない．

目的：形式スケッチ中の行間を埋める局所補完モデルを構築する．

3. タスク定義 と データ構築

入力：Sketch（行間を含む部分的な形式証明） → 出力：補完片（行間を置換する コード片）

Sketch :

```
theorem ex1 (x y : ℕ) (hx : x ≤ 3) (hy : y ≤ 4) (hxy : 7 ≤ x + y) : x = 3 ∧ y = 4 := by
  have hx_eq : x = 3 := <proof>
  have hy_eq : y = 4 := <proof>
  exact ⟨hx_eq, hy_eq⟩
```

行間 → 補完片 : (by omega , by omega)

検証済み Lean 4 証明 → 形式スケッチ sketch / 補完片

5. 推論手法 モデルの補完対象を 1 個に特定

sorry： Lean 4 には，証明が未完成であることを明示し，この部分の証明をとりあえず認めて，検証を継続させるためのプレースホルダである．

順次補完

先に生成した補完片が，後続の穴を補完する際の入力文脈に含まれるため，先行する補完片が後続の推論の結果に影響を与える可能性がある．

形式スケッチ

```
theorem ex1 (x y : ℕ) (hx : x ≤ 3) (hy : y ≤ 4) (hxy : 7 ≤ x + y) : x = 3 ∧ y = 4 := by
  have hx_eq : x = 3 := <proof>
  have hy_eq : y = 4 := <proof>
  exact ⟨hx_eq, hy_eq⟩
```

補完する穴以外を sorry で

1. 穴 1 を補完

```
theorem ex1 … := by
  have hx_eq : x = 3 := <proof>
  have hy_eq : y = 4 := sorry
  exact ⟨hx_eq, hy_eq⟩
```

model → 補完片 f_1 = by omega を生成し，挿入

1. 穴 2 を補完

```
theorem ex1 … := by
  have hx_eq : x = 3 := by omega
  have hy_eq : y = 4 := <proof>
  exact ⟨hx_eq, hy_eq⟩
```

model → 補完片 f_2 = by omegaを生成し，挿入

完全な形式証明

```
theorem ex1 (x y : ℕ) (hx : x ≤ 3) (hy : y ≤ 4) (hxy : 7 ≤ x + y) : x = 3 ∧ y = 4 := by
  have hx_eq : x = 3 := by omega
  have hy_eq : y = 4 := by omega
  exact ⟨hx_eq, hy_eq⟩
```

並列補完

他の穴の補完結果が文脈として入力されないため，生成段階における順序依存性は生じない．

形式スケッチ

```
theorem ex1 (x y : ℕ) (hx : x ≤ 3) (hy : y ≤ 4) (hxy : 7 ≤ x + y) : x = 3 ∧ y = 4 := by
  have hx_eq : x = 3 := <proof>
  have hy_eq : y = 4 := <proof>
  exact ⟨hx_eq, hy_eq⟩
```

```
theorem ex1 … := by
  have hx_eq : x = 3 := <proof>
  have hy_eq : y = 4 := sorry
  exact ⟨hx_eq, hy_eq⟩
```

補完片 f_1 を生成 → model → f_1 = by omega

```
theorem ex1 … := by
  have hx_eq : x = 3 := sorry
  have hy_eq : y = 4 := <proof>
  exact ⟨hx_eq, hy_eq⟩
```

補完片 f_2 を生成 → model → f_2 = by omega

f_1 , f_2 を一括で挿入 → 完全な形式証明

```
theorem ex1 (x y : ℕ) (hx : x ≤ 3) (hy : y ≤ 4) (hxy : 7 ≤ x + y) : x = 3 ∧ y = 4 := by
  have hx_eq : x = 3 := by omega
  have hy_eq : y = 4 := by omega
  exact ⟨hx_eq, hy_eq⟩
```

6. 実験 と 考察

ベースモデル： DeepSeek-Prover-V2-7B
ベンチマーク： FormalMATH，MiniF2F
評価指標： pass@k

pass@k : 各サンプルについて最大 k 個の候補を生成し，少なくとも1個が Lean 4 検証を通過すれば正解とする

表1：データ構築結果

ベンチマーク	生成証明数	Lean 検証通過数	Sketch数	定理数	平均穴数
FormalMATH	55,600	5,980	3,616	563	2.82
MiniF2F	4,880	2,850	1,842	260	2.34
合計	60,480	8,830	5,458	823	2.66

表2：順次補完，並列補完の評価結果

ベンチマーク	pass@1	pass@12	pass@24	pass@32
順次補完				
FormalMATH	80.03%	93.40%	95.49%	95.97%
MiniF2F	83.23%	92.22%	93.11%	93.41%
合計	81.15%	92.98%	94.66%	95.08%
並列補完				
FormalMATH	73.75%	89.05%	91.63%	92.75%
MiniF2F	80.54%	93.41%	94.61%	94.61%
合計	76.13%	90.58%	92.67%	93.40%

表3：ポイント差分（順次－並列）

ベンチマーク	Δpass@1	Δpass@12	Δpass@24	Δpass@32
FormalMATH	+6.28	+4.35	+3.86	+3.22
MiniF2F	+2.69	−1.19	−1.50	−1.20
合計	+5.02	+2.40	+1.99	+1.68

表4：穴数別の pass@k（全体，単一穴／複数穴）

穴数バケット	手法	pass@1	pass@12	pass@24	pass@32
単一穴 (375)	共通	86.40%	96.80%	98.13%	98.93%
複数穴 (580)	順次	77.76%	90.52%	92.41%	92.59%
	並列	69.48%	86.55%	89.14%	89.83%

→ 先行して補完したコードが後続の補完タスクの文脈として与えることが補完の精度向上に寄与している．

→ 小さい k で証明できない難しい命題に対しては，先行する穴に対する「もっともらしいが，誤った補完コード」が後続の補完を誤った方向へ誘導してしまう可能性がある．